Rapport de projet

Avant-propos

Ce rapport a été écrit au format Markdown puis converti en PDF avec pandoc. Toutefois, pour pouvoir suivre les liens vers les fichiers locaux, il est préférable de s'en référer à la version originale au format Markdown. Le modèle des classes se trouve en toute fin du rapport.

MVC

Le jeu adopte une architecture MVC. La répercussion des changements de la vue et du modèle est implémentée via des `PropertyChangeListener` ajoutés à chaque vue/modèle supervisé par un contrôleur. Vue et modèle peuvent ensuite utiliser `firePropertyChange`

Vue Notre implémentation supporte deux types d'affichage, le dessin à l'aide d'une instance de `Graphics2D` (cf. `AbstractView.draw`) et/ou l'ajout de `Component` au `JPanel` principale via `JPanel.add(Component comp)` (ou autre surcharge). `AbstractView` prend automatiquement en charge l'affichage des éléments renvoyer par `getUIComponents`. Ainsi, les sous-classes peuvent juste surcharger cette fonction de sorte à ce qu'elle renvoyer les éléments à afficher ou à cacher.

Représentation des objets du jeu

Une classe nommée `RootObject` est utilisée pour regrouper les informations de base nécéssaires pour stocker et représenter un objet du jeu sans qu'il fasse partie de la carte. Si un objet est destiné à être possédé par le joueur, `targetedClass` doit être non-nul (cf. `constructEntity`).

Inventaires

Tous les inventaires sont dérivés d'une super classe `InventoryView`. Celle-ci peut être customisé via ces fonctions d'affichage pour chaque composant du menu d'inventaire. Ces fonctions sont accessibles aux sous-classes de `InventoryView` (cf. `PlayerInventoryView`, `FactoryInventoryView`, ...).

`InventoryView` prend en paramètre un nombre variable d'inventaires de telle sorte que chaque inventaire aura une section dédiée.

Gestions des états du jeu

Les différents états/vues du jeu sont stockés à l'aide d'une pile dans `GameStateController`. Lorsqu'un nouveau contrôleur est ajouté au sommet de la pile, on l'affiche automatiquement à l'écran via un appel à `AbstractController.activate`.

Implémentation du jeu

1. Ecran d'acceuil

L'écran d'accueil s'ouvre en appuyant sur la touche ESC (ou Echap). Lorsque le joueur appuie sur cette touche l'état de jeu est changé dans `GameState`. En effet, on push l'état de jeu 'MENU' dans la Stack évoquée précedemment. Ainsi, un écran de menu apparait avec 4 boutons :

- LOAD : L'action associée au bouton est le chargement d'une partie sauvegardée.
- NEW GAME : Pour ce second bouton, le joueur commence une nouvelle partie vierge.
- SETTINGS : Une page de paramètres s'affiche à l'écran.
- EXIT : On demande au joueur s'il veut sauvegarder sa partie avant de quitter.

La classe contrôlant les diverses choix du joueur est `MenuController`

Sauvegarde de la partie La sauvegarde de la partie est effectuée dans le fichier txt `save.txt`. Ce fichier contient dans l'ordre :

- Les informations du joueur (## PLAYER DATA), qui sont sa position X;Y de départ puis son argent accumulé ce qui nous donne X;Y;ARGENT
- Les informations de la map (## MAP DATA) qui est la matrice de la carte, il s'agit seulement des positions de l'eau, de la pelouse et du sable.
- Les informations concernant les objets de la carte (## MAP OBJECT DATA). Il s'agit ici des objets de la carte au moment de la sauvegarde (position des ressources, machines, marché...).
- Les informations de l'inventaire du joueur (## INVENTORY ITEM DATA). On enregistre un couple identifiant;quantité pour chaque objet.

Pour sauvegarder une partie il faut ouvrir le menu puis selectionner le bouton EXIT pour ensuite cliquer sur le bouton OUI.

2. Ecran de jeu

L'écran de jeu est affiché lorsque la partie est dans l'état de jeu `PLAYING`. De ce fait, il est contrôlé par `PlayingController`. Ce contrôleur se charge de déleguer les actions à des contrôleurs comme `PlayerController` et `MapController`.

Ainsi l'écran de jeu affiche le joueur et la carte sur laquelle ce dernier peut se déplacer, effectuer des actions, etc.

Génération de la carte La carte est généré pseudo-aléatoirement dans `MapModel` par des méthodes comme `generateMapData`, `createClusters`, `createPaths`, `createMap`. Ces méthodes essaient de produire une carte avec une certaine harmonie parmi le hasard. Cela permet de ne pas avoir des carreaux de sable, d'eau et de pelouse disposés de manière complétement aléatoire. Ensuite pour la génération des objets de la carte (ressources, marché), la méthode createMap utilise un objet de la classe Random et on contrôle l'apparition des ressources pour ce qu'elles soient distribués de manière égale. Concernant la case du marché, elle est toujours situé sur les bords encore une fois pseudo-aléatoirement.

3. Actions du joueur dans l'écran de jeu

3.1 Déplacements Les déplacements du joueurs sont limités aux touches directionnelles du clavier. Le joueur est bloqué s'il souhaite sortir de la carte. Lorsque ce dernier se déplace, une animation se met en route pour que son déplacement paraisse naturel. Des booleéns up, down, left, right sont inclus dans la classe `PlayerModel` pour connaitre la touche qui a été enfoncée.

3.2 Récoltes de ressources Pour récolter une ressource, le joueur peut cliquer sur celle-ci. L'état actuel de la ressource est indiqué via une pastille verte lorsqu'elle peut être récoltée. Autrement, une barre de progression est affichée.

La récolte d'une ressource est "temporisée" et peut en plus être "laborieuse" en fonction du niveau de "rareté" de celle-ci. La condition bloquante introduite par la récolte "laborieuse" est honorée dans `PlayerModel`. En effet, lorsque le joueur clique sur une ressource et que celle-ci est récoltable, un jeton est passé au joueur et en fonction du type de récolte stocké dans le jeton, les mouvements du joueur sont ignorés si nécessaire. Ensuite, on effectue périodiquement un appel à `ResourceHarvestToken.harvest` dans `PlayerModel.checkPendingResource` pour pouvoir ajouter la ressource à notre inventaire si le temps de récolte est écoulé.

4. L'écran d'inventaire et de craft (via la touche espace)

Cette inventaire contient deux sections, "Resources" et "Craft". La vue est implémentée par `PlayerInventoryView` et l'affichage est géré par `PlayerController`. La première section ne diffère en rien de l'affichage défini par défaut dans `InventoryView`. Le joueur peut uniquement consulter le contenu de son inventaire.

La section “Craft” affiche l’ensemble des recettes existantes avec les ingrédients requis et le temps de préparation. Le joueur peut exécuter une recette à condition d’avoir l’ensemble des ingrédients et que celle-ci ne soit pas en cours d’exécution. Comme précédement, le caractère “instantanées” ou “temporisées” de chaque recette dépend du facteur de rareté de la ressource finale. 6 recettes sont disponibles, chacune ayant nécessitant des ingrédients plus ou moins rares.

5. L’écran du marché

Le marché comporte deux sections, “vente” et “achat”, ainsi que la balance du joueur. L’inventaire pour la section “achat” est initialisé avec l’ensemble des objets constructibles (cela signifie que la variable `RootObject.targetedClass` est non nulle) du jeu et en nombre infini. L’inventaire pour la section vente est juste un pointeur vers l’inventaire du joueur. Toutefois, pour pouvoir afficher un titre de section différent, une sous-classe de `InventoryModel` était nécessaire. Sa seule utilité est de surcharger la fonction `getTitle`. Mis à part cela, les appels aux autres fonctions sont transmis à l’inventaire du joueur.

Une variante XXL est disponible pour l’usine et la récolteuse. Ici, rien de bien compliqué, il suffit d’avoir une sous-classe héritant des machines respectives qui configure une capacité d’inventaire plus élevée.

6. Machines

Deux types de machine sont disponibles, l’usine et la récolteuse. Ces deux machines vont périodiquement essayer de compléter leur tâche respective, à savoir exécuter une recette pour l’usine, et récolter une ressource pour la récolteuse. Une fois la tâche terminée, leur inventaire est mis à jour et on recommence.

Configuration Une machine est initialement à l’arrêt car non configurée. Lorsque le joueur pose une machine, une pastille rouge indiquant que la machine doit être configurée est affichée. Ainsi, en cliquant sur une machine non configurée, le menu de configuration de celle-ci s’affiche. Autrement, on affiche l’inventaire de la machine et le menu de configuration est disponible via un bouton en haut à gauche de l’inventaire.

7. Inventaire d’une machine

Pour accéder à l’inventaire d’une machine (configurée préalablement), il suffit de cliquer sur celle-ci. Toutefois, pour interagir avec un objet du jeu, le joueur doit se trouver dans une case adjacente à celle-ci.

7.1 Inventaire d’une récolteuse La récolteuse n’ayant qu’un inventaire (l’inventaire hérité de MachineModel), seul son stock est affiché. Le joueur peut ensuite ajouter une ou l’ensemble des unités d’une ressource dans son propre inventaire.

7.2 Inventaire d’une usine L’usine contient deux inventaires, un hérité de MachineModel pour son stock et un autre pour ses ingrédients. Dans la section ingrédients, le joueur peut transférer des ressources de son inventaire vers celui de l’usine et inversement. Dans la section stock, le joueur peut récupérer les ressources produites par l’usine.

Possible améliorations

Actuellement, le design graphique des inventaires est très basique. Une refonte graphique de ceux-ci semble donc souhaitable. De plus, lorsque la partie graphique d’un inventaire est mise à jour, la position de la barre de défilement (scroll) n’est pas préservée. De la même façon que l’indice de l’onglet sélectionné est préservé, il faudra en faire de même pour la barre de défilement.

Il serait également bien d’ajouter un peu plus de dynamisme dans le jeu. Par exemple, une bande-son ou bien remplacer les pastilles pour indiquer les statuts d’une ressource par des icônes ou encore animer les ressources (différentes étapes de la pousse, récolte laborieuse, etc).

Peut-être un peu ambitieux, mais un mode multijoueur peut être une bonne idée pour une grosse amélioration. Ainsi, plusieurs joueurs pourraient gérer les ressources, se les échanger ou se les vendre et même former des équipes pour effectuer plus vite des récoltes laborieuses comme les pépites d'or actuellement.

Modèle des classes

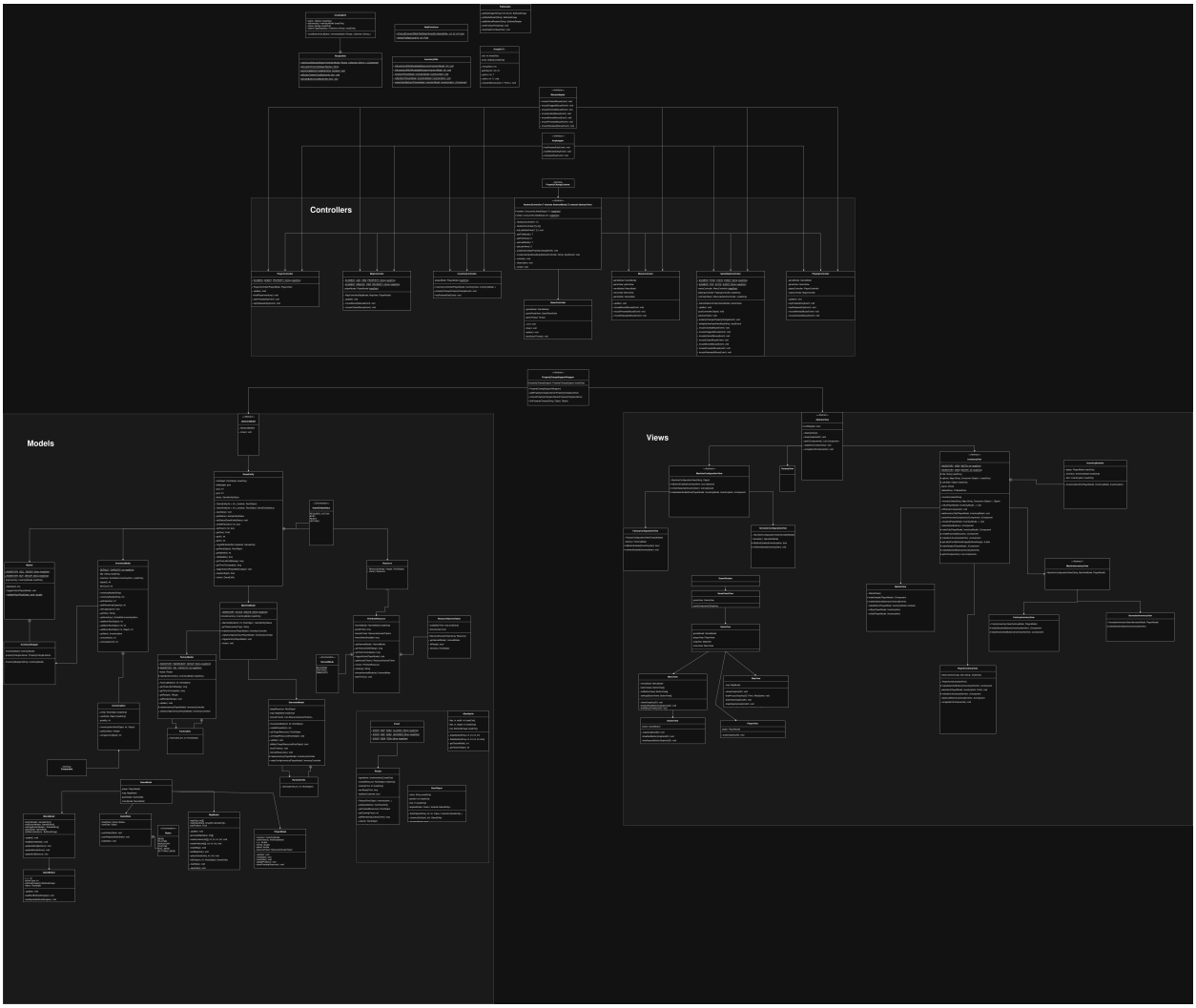

Figure 1: UML